
ANYTHING2SKILL: COMPILING EXTERNAL KNOWLEDGE INTO REUSABLE SKILLS FOR AGENTS

Qianjun Pan¹, Yutao Yang¹, Junsong Li¹, Jie Zhou^{1,2*}, Kai Chen², Xin Li², Qin Chen¹, Liang He¹

¹ School of Computer Science and Technology, East China Normal University, Shanghai

² Shanghai AI Laboratory

{jzhou, qchen, lhe}@cs.ecnu.edu.cn

<https://github.com/ECNU-ICALK/AutoSkill>

June 23, 2026

ABSTRACT

Retrieval-augmented generation (RAG) enables agents to access external knowledge at inference time, but it primarily retrieves fragmented declarative evidence, leaving agents to repeatedly infer task procedures from passages, manuals, examples, logs, or trajectories. This raises a fundamental question: **can skills extracted from external knowledge bases be installed into an agent, enabling it to rapidly approximate domain expertise?** In this paper, we propose ANYTHING2SKILL, a taxonomy-guided framework that compiles heterogeneous external knowledge into reusable, retrievable, and executable skills for agents. Given a corpus of knowledge records, ANYTHING2SKILL first decomposes each record into evidence windows and performs plan-and-expand skill extraction under a skill-tree prior. The extracted candidates are then converted into structured skill contracts that specify invocation conditions, contraindications, action moves, workflow steps, constraints, output specifications, supporting evidence, and confidence scores. To construct a deployable procedural memory, ANYTHING2SKILL manages the extracted skills in a persistent SkillBank through taxonomy-aware compilation, registry-level reconciliation, lifecycle tracking, versioned updates, and visible skill-tree projection. At inference time, agents retrieve both task-specific passages from the original knowledge base and relevant procedural skills from the SkillBank, allowing RAG to provide declarative evidence while compiled skills provide reusable procedural guidance. Experiments on qsv and GitHub-CLI show that ANYTHING2SKILL combined with RAG achieves 98.85% and 94.10% success rates, respectively, substantially outperforming RAG-only agents. These results suggest that compiling latent procedural knowledge into explicit skills is an effective way to extend retrieval-augmented agents from knowledge access toward capability reuse.

Keywords Agentic systems · Procedural knowledge · Skill compilation · Procedural memory · Retrieval-augmented generation

1 Introduction

Large language models (LLMs) are increasingly deployed as autonomous agents that reason over external information, invoke tools, interact with environments, and complete multi-step tasks. A central challenge for such agents is how to acquire and reuse knowledge beyond the parametric memory of the underlying model. Retrieval-augmented generation (RAG) has emerged as a widely adopted solution by enabling models to access external, updateable, and interpretable knowledge at inference time [Lewis et al., 2020, Guu et al., 2020, Karpukhin et al., 2020]. By retrieving relevant documents, passages, web pages, or API descriptions, RAG substantially improves factual grounding and supports knowledge-intensive reasoning across open-domain question answering, long-form generation, and tool-use scenarios [Izacard and Grave, 2021, Borgeaud et al., 2022, Izacard et al., 2023, Asai et al., 2024].

*Corresponding Author

Despite its effectiveness, existing RAG systems primarily treat external knowledge as declarative evidence. Retrieved passages tell the agent what information is relevant, but they usually do not directly specify how a task should be performed. As a result, agents must repeatedly reconstruct procedural knowledge from fragmented evidence, such as manuals, examples, dialogue histories, execution logs, or task trajectories. This limitation is especially salient in agentic settings, where success often depends not only on retrieving the right facts, but also on following the right procedures: when to invoke a tool, which action sequence to execute, what constraints to satisfy, what intermediate artifacts to produce, and which failure cases to avoid. Even advanced retrieval-augmented reasoning methods that interleave retrieval with generation or reflection [Trivedi et al., 2023, Jiang et al., 2023, Khattab et al., 2022, Asai et al., 2024] still require the model to infer such procedures at task time, rather than storing them as reusable capabilities.

In parallel, recent work on LLM agents has shown that explicit skills, tools, memories, and executable routines are crucial for long-horizon problem solving Yang et al. [2026]. ReAct interleaves reasoning and acting to improve interactive decision making [Yao et al., 2023a]; Toolformer, Gorilla, ToolLLM, and related systems improve the ability of LLMs to call external APIs and tools [Schick et al., 2023, Patil et al., 2024, Qin et al., 2024]; Reflexion and Generative Agents demonstrate the value of externalized memory and self-reflection for agent improvement [Shinn et al., 2023, Park et al., 2023]. In embodied and open-ended environments, skill libraries and reusable policies have also been shown to support long-horizon planning and continual exploration [Ahn et al., 2022, Liang et al., 2022, Wang et al., 2023]. However, most existing skill-based agents either rely on manually specified skills, skills learned through environment interaction, or executable code generated during task solving. They do not directly address a complementary and practical source of capabilities: the large amount of latent procedural knowledge already embedded in heterogeneous external knowledge bases.

In this paper, we propose ANYTHING2SKILL, a taxonomy-guided framework that compiles heterogeneous external knowledge into reusable, retrievable, and executable skills for LLM agents. Instead of using external knowledge only as task-time context, ANYTHING2SKILL transforms documents, manuals, dialogues, logs, and trajectories into structured skill contracts. Each skill contract specifies the skill name, description, normalized asset type, taxonomy node, invocation conditions, contraindications, action moves, workflow steps, constraints, output specifications, supporting evidence, and confidence scores. These skills are stored in a persistent SkillBank, which serves as procedural memory for downstream agents.

The key idea behind ANYTHING2SKILL is to bridge retrieval-augmented knowledge access and skill-based capability reuse. Given a heterogeneous knowledge corpus, our framework first decomposes each record into evidence windows and performs plan-and-expand skill extraction under a skill-tree prior. The skill-tree prior constrains extraction toward procedural, evidence-grounded capabilities rather than free-form summaries. Extracted candidates are then normalized into stable asset types and converted into structured skill drafts. To make the resulting skills deployable, ANYTHING2SKILL further manages them in a SkillBank through taxonomy-aware compilation, registry-level reconciliation, lifecycle tracking, versioned updates, and visible skill-tree projection. At inference time, an agent retrieves both task-specific evidence from the original knowledge base and relevant procedural skills from the SkillBank. In this way, RAG provides declarative grounding, while the SkillBank provides reusable procedural guidance.

We evaluate ANYTHING2SKILL on agent tasks involving qsv and GitHub-CLI. Experimental results show that combining ANYTHING2SKILL with RAG achieves success rates of 98.85% and 94.10%, respectively, substantially outperforming RAG-only agents. These results indicate that many useful agent capabilities can be recovered from existing knowledge sources when implicit procedures are compiled into explicit, reusable skills. More broadly, our findings suggest that retrieval-augmented agents should not only retrieve external knowledge, but also organize and reuse the procedural capabilities latent within that knowledge.

Our contributions are summarized as follows:

- We propose ANYTHING2SKILL, a taxonomy-guided framework that extends retrieval-augmented agents from external knowledge access to reusable capability acquisition by compiling heterogeneous knowledge records into structured, evidence-grounded skills.
- We implement a complete skill compilation and management pipeline, including evidence-window construction, plan-and-expand skill extraction under a skill-tree prior, structured skill-contract generation, taxonomy-aware consolidation, registry-level reconciliation, lifecycle tracking, versioned updates, and visible SkillBank projection.
- We empirically demonstrate that augmenting RAG agents with the compiled SkillBank substantially improves task success on qsv and GitHub-CLI benchmarks, achieving 98.85% and 94.10% success rates, respectively, and consistently outperforming RAG-only agents.

2 Related Work

2.1 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) has become a standard paradigm for equipping language models with external, updateable, and interpretable knowledge. Early open-domain question answering systems combined retrieval with neural reading models, retrieving evidence from large corpora before extracting or generating answers [Chen et al., 2017, Lee et al., 2019, Seo et al., 2019]. Dense Passage Retrieval (DPR) further demonstrated that learned dense retrievers can substantially improve open-domain evidence retrieval over sparse lexical baselines [Karpukhin et al., 2020], while REALM introduced retrieval into language model pre-training by learning a latent retriever over a large textual corpus [Guu et al., 2020]. RAG formalized the integration of a parametric sequence-to-sequence generator with a non-parametric memory accessed through dense retrieval, showing strong performance on knowledge-intensive NLP tasks [Lewis et al., 2020]. Subsequent work improved the reader-generator component, such as Fusion-in-Decoder, which encodes multiple retrieved passages independently and fuses them during decoding [Izacard and Grave, 2021].

A second line of work studies retrieval as a scalable memory mechanism for language models. RETRO conditions autoregressive generation on chunks retrieved from a large-scale database, showing that retrieval can reduce the need to store all knowledge in model parameters [Borgeaud et al., 2022]. Atlas demonstrates that retrieval-augmented language models can support strong few-shot learning on knowledge-intensive tasks with an updateable document index [Izacard et al., 2023]. REPLUG explores retrieval augmentation for black-box language models by prepending retrieved documents to the input and using the language model to supervise the retriever [Shi et al., 2024]. Other work investigates when parametric knowledge is sufficient and when non-parametric retrieval is necessary [Mallen et al., 2023], as well as benchmarks for evaluating knowledge-intensive language tasks [Petroni et al., 2021]. Retrieval quality itself has also been improved through sparse-dense retrieval, late interaction, and unsupervised dense retrieval methods, including BM25-style probabilistic retrieval [Robertson and Zaragoza, 2009], ColBERT [Khattab and Zaharia, 2020], Contriever [Izacard et al., 2022], and HyDE [Gao et al., 2023a].

Recent RAG research moves beyond a single retrieve-then-generate pipeline. WebGPT retrieves and browses web evidence to support long-form question answering with human feedback [Nakano et al., 2021]. IRCOT interleaves retrieval with chain-of-thought reasoning, allowing intermediate reasoning steps to guide subsequent retrieval for multi-hop questions [Trivedi et al., 2023]. Demonstrate-Search-Predict formulates retrieval-augmented in-context learning as a compositional pipeline over language-model and retrieval-model calls [Khattab et al., 2022]. FLARE actively decides when and what to retrieve during generation, especially for long-form outputs [Jiang et al., 2023]. Self-RAG further trains models to decide when to retrieve, how to use retrieved passages, and how to critique their own generations through reflection tokens [Asai et al., 2024]. These methods improve the timing, grounding, and controllability of retrieval.

Despite these advances, most RAG systems still treat external knowledge as declarative evidence: passages, documents, examples, web pages, or API descriptions are retrieved and then interpreted at inference time. This design is effective for factual grounding, but it leaves the agent to repeatedly reconstruct procedures from fragmented context. In contrast, ANYTHING2SKILL focuses on the procedural knowledge implicitly embedded in heterogeneous knowledge bases. Rather than only retrieving passages, it compiles documents, manuals, logs, conversations, and trajectories into structured, evidence-grounded skills that can be stored, managed, retrieved, and reused. In this sense, our work complements RAG: retrieval provides task-time declarative evidence, while skill compilation provides reusable procedural memory.

2.2 Agent Skills

Large language models are increasingly used as agents that reason, call tools, interact with environments, and execute multi-step tasks. ReAct shows that interleaving reasoning traces with actions improves both question answering and interactive decision making [Yao et al., 2023a]. Toolformer trains language models to decide when and how to call external tools using self-supervised API-use data [Schick et al., 2023]. MRKL systems propose a modular neuro-symbolic architecture in which language models route problems to external tools, knowledge sources, or symbolic modules [Karpas et al., 2022]. HuggingGPT uses an LLM as a controller to plan tasks, select models from Hugging Face, execute subtasks, and summarize results [Shen et al., 2023]. Gorilla improves API calling by combining fine-tuning with retrieval over changing API documentation [Patil et al., 2024]. ToolLLM and ToolBench scale tool-use training and evaluation to thousands of real-world APIs [Qin et al., 2024], while API-Bank provides a benchmark for tool-augmented LLMs [Li et al., 2023a]. Program-aided language models and program-of-thought prompting similarly externalize computation into executable programs [Gao et al., 2023b, Chen et al., 2023].

Another line of work studies agents that improve through feedback, memory, planning, and interaction. Reflexion enables language agents to use verbal feedback and maintain reflective memory without updating model weights [Shinn et al., 2023]. Generative Agents simulate human-like behavior by storing and retrieving memory records for planning and reflection [Park et al., 2023]. CAMEL studies communicative multi-agent role-playing as a way to elicit cooperative problem solving among language agents [Li et al., 2023b]. Tree of Thoughts extends prompting with search over intermediate reasoning states [Yao et al., 2023b]. These works demonstrate that agents benefit from externalized reasoning traces, memories, and feedback. However, their memories are often episodic, task-specific, or unstructured, whereas our work aims to compile reusable procedural skills from heterogeneous prior knowledge.

Skill-based agents have also been widely explored in robotics, embodied AI, and reinforcement learning. SayCan grounds language-model planning in pretrained robotic skills and affordance scores, enabling feasible long-horizon robotic behavior [Ahn et al., 2022]. Inner Monologue uses natural-language feedback from the environment to support embodied reasoning and planning [Huang et al., 2022a]. Code as Policies generates executable robot policy code from language instructions [Liang et al., 2022]. ProgPrompt, LM-Nav, PaLM-E, RT-2, and language-model zero-shot planners further show that language models can compose actions, policies, and embodied knowledge for robotics and navigation [Singh et al., 2023, Shah et al., 2023, Driess et al., 2023, Brohan et al., 2023, Huang et al., 2022b]. In open-ended environments, Voyager maintains an ever-growing library of executable code skills and reuses them for lifelong exploration in Minecraft [Wang et al., 2023]. More broadly, reinforcement learning has studied skill discovery, option learning, and skill priors as mechanisms for temporal abstraction and generalization [Eysenbach et al., 2019, Pertsch et al., 2021, Andreas et al., 2017], while program synthesis work such as DreamCoder learns reusable libraries that accelerate future problem solving [Ellis et al., 2021]. Recent work also explores inducing and planning with latent language skills [Sharma et al., 2022].

These studies show that reusable skills are crucial for long-horizon agency, tool use, embodied control, and continual improvement. However, most existing agent-skill systems either assume that skills are manually specified, learned through environment interaction, distilled from execution attempts, or stored as code generated during task solving. ANYTHING2SKILL addresses a complementary setting: many useful procedures already exist implicitly in documents, manuals, dialogues, logs, and trajectories, but they are not directly available as reusable agent skills. Our framework extracts such latent procedural knowledge, grounds it in source evidence, normalizes it with a skill taxonomy, and manages it in a persistent SkillBank through compilation, reconciliation, versioning, and hierarchy projection. This allows agents to retrieve not only task-relevant content, but also reusable procedural capabilities compiled from external knowledge.

3 Method

3.1 Overview

Retrieval-augmented generation enables agents to access external knowledge at inference time, but it primarily exposes agents to fragmented declarative evidence, leaving them to repeatedly reconstruct task procedures from passages, examples, manuals, logs, or trajectories. ANYTHING2SKILL addresses this limitation by compiling heterogeneous external knowledge into reusable, retrievable, and executable skills (Figure 1). Given a corpus of knowledge records $\mathcal{K} = \{D_i\}_{i=1}^N$, where each D_i may be a document, dialogue, manual, execution log, or trajectory, our goal is to discover latent procedural knowledge that specifies how tasks should be performed, when tools or actions should be invoked, what constraints should be followed, and what outputs should be produced. The compiled skills are stored in a persistent SkillBank $\mathcal{B}$ and can be retrieved at inference time to guide agent execution, thereby complementing RAG with procedural capabilities distilled from external knowledge bases.

Formally, ANYTHING2SKILL performs a taxonomy-guided compilation process:

$$\mathcal{B} = \text{ANYTHING2SKILL}(\mathcal{K}, \mathcal{T}), \quad (1)$$

where $\mathcal{T}$ is a skill taxonomy that provides a structural prior over skill asset types, hierarchy levels, and parent-child relations. The method consists of two main components. First, *skill extraction with a skill-tree prior* identifies evidence-grounded procedural knowledge from heterogeneous records and converts it into structured skill drafts. Second, *skill management in the SkillBank* consolidates, reconciles, versions, and organizes extracted skills into a coherent repository for later retrieval and execution.

Each skill s is represented as a structured contract:

$$s = \langle n, d, a, v, g, \tau, \sigma^+, \sigma^-, \mathbf{m}, \mathbf{p}, \mathbf{c}, \mathbf{o}, \mathcal{E}, \rho \rangle, \quad (2)$$

where n and d are the skill name and description; a is the normalized asset type; v is the taxonomy node; g is the granularity level; τ stores contextual metadata such as domain, family, task type, method type, and stage; σ^+ and

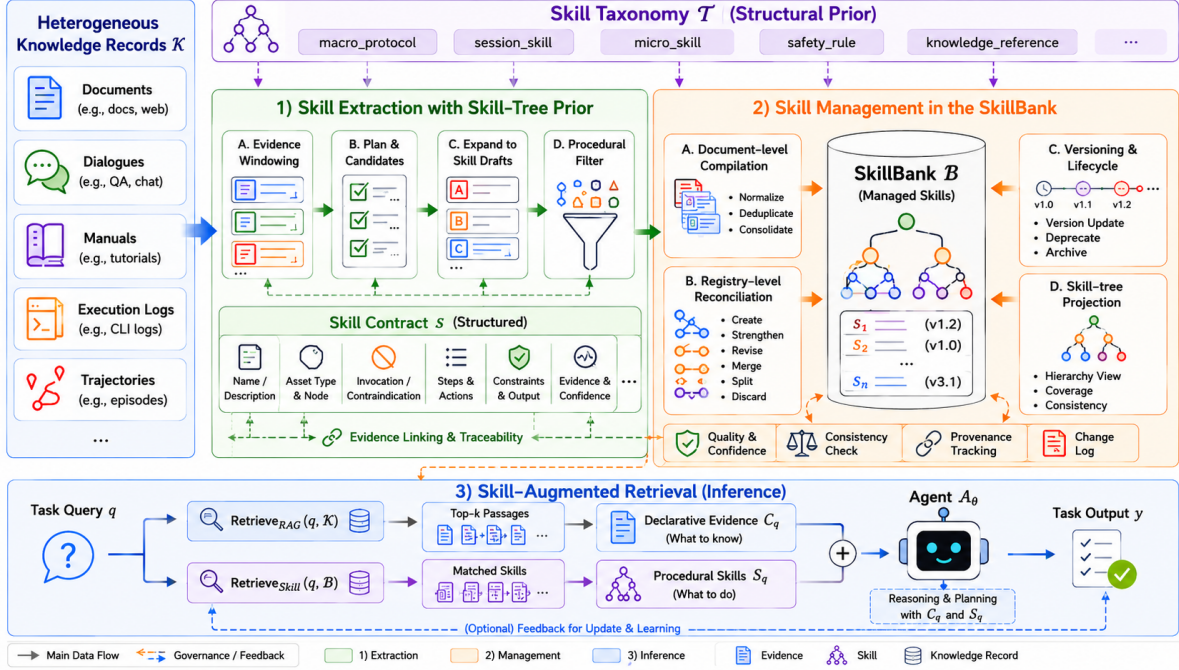


Figure 1: Overview of ANYTHING2SKILL, which compiles heterogeneous knowledge records into reusable procedural skills. The framework uses a skill taxonomy as a structural prior for skill extraction, manages extracted skills through compilation, reconciliation, and versioned tree projection, and supports inference by retrieving both declarative evidence and procedural skills for agent execution.

σ^- denote invocation and contraindication conditions; $\mathbf{m}$ denotes action or intervention moves; $\mathbf{p}$ denotes workflow steps; $\mathbf{c}$ denotes constraints and cautions; $\mathbf{o}$ denotes the output contract; $\mathcal{E}$ is the supporting evidence set; and ρ is the confidence score.

3.2 Skill Extraction with Skill-Tree Prior

Skill taxonomy. The skill taxonomy $\mathcal{T}$ defines the structural prior used to guide extraction and later management. We define

$$\mathcal{T} = (\mathcal{A}, \mathcal{V}, \mathcal{R}, \pi, \ell, \mathcal{F}), \quad (3)$$

where $\mathcal{A}$ is the set of stable internal asset types, $\mathcal{V}$ is the set of taxonomy nodes, $\mathcal{R} \subseteq \mathcal{V} \times \mathcal{V}$ defines parent-child relations, $\pi : \mathcal{V} \rightarrow \mathcal{A}$ maps each taxonomy node to an asset type, $\ell : \mathcal{V} \rightarrow \mathbb{N}$ assigns a hierarchy level, and $\mathcal{F}$ stores domain- or family-specific buckets.

The internal asset type space is intentionally compact:

$$\mathcal{A} = \{\text{macro_protocol}, \text{session_skill}, \text{micro_skill}, \text{safety_rule}, \text{knowledge_reference}\}. \quad (4)$$

Domain-specific aliases are normalized into this stable type space through a taxonomy-specific normalizer:

$$a = N_{\mathcal{T}}(\tilde{a}), \quad (5)$$

where $\tilde{a}$ is the raw asset label produced by the extractor. If $N_{\mathcal{T}}(\tilde{a}) = \perp$, the candidate is rejected.

Given a candidate skill $\tilde{s}$ and its normalized asset type a , its taxonomy node is resolved as

$$v = \arg \max_{v' \in \mathcal{V} : \pi(v') = a} \text{match}(\tilde{s}, v'), \quad (6)$$

where $\text{match}(\cdot)$ measures compatibility between the candidate and a taxonomy node using node labels, aliases, hierarchy constraints, and family metadata. This taxonomy prior constrains the extractor to produce procedural skills rather than free-form summaries, and prevents incompatible skill types from being merged in later management.

Evidence windows. Each input record D_i is first normalized and decomposed into strict evidence windows:

$$W_i = \{w_{i1}, w_{i2}, \dots, w_{im_i}\}. \quad (7)$$

Each window is represented as

$$w_{ij} = \langle x_{ij}, h_{ij}, r_{ij}, b_{ij}, m_{ij} \rangle, \quad (8)$$

where x_{ij} is the text span, h_{ij} is the local heading context, r_{ij} denotes the original paragraph or character span, b_{ij} records boundary and anchor information, and m_{ij} stores metadata such as source path, heading path, sibling headings, and subsection information. This design ensures that every extracted skill can be traced back to explicit source evidence.

Plan-and-expand extraction. Directly asking a model to extract all skills from a long document often produces redundant, underspecified, or summary-like outputs. ANYTHING2SKILL therefore uses a two-stage plan-and-expand extraction strategy. In the planning stage, an LLM-based planner identifies distinct skill candidates from each evidence window:

$$\mathcal{P}_{ij} = P_\theta(w_{ij}, \mathcal{T}, K), \quad (9)$$

where K is the maximum number of candidates per window. Each planned candidate $p_{ijk} \in \mathcal{P}_{ij}$ contains a lightweight objective, tentative asset type, taxonomy node, invocation cues, contraindications, expected artifact, evidence hints, and confidence.

In the expansion stage, each candidate is independently expanded into a structured skill draft:

$$\tilde{s}_{ijk} = E_\theta(w_{ij}, p_{ijk}, \mathcal{T}). \quad (10)$$

The expansion is conservative: if the window does not directly support the planned procedure, the extractor returns a null draft rather than hallucinating unsupported steps.

To distinguish procedural skills from ordinary factual descriptions, we apply a procedural validity filter:

$$I_{\text{proc}}(\tilde{s}) = \mathbb{I}[|\mathbf{p}| + |\mathbf{m}| + |\mathbf{c}| > 0] \cdot \mathbb{I}[N_{\mathcal{T}}(\tilde{a}) \neq \perp]. \quad (11)$$

A candidate is retained only if it contains at least one procedural component, such as workflow steps, action moves, constraints, or cautions, and can be mapped into the taxonomy.

For each retained draft, ANYTHING2SKILL creates a support record:

$$e_{ijk} = \langle \text{id}(D_i), r_{ij}, \text{excerpt}(w_{ij}), q_{ijk}, \rho_{ijk} \rangle, \quad (12)$$

where q_{ijk} denotes the support relation type, such as support, constraint, conflict, or case variant. The extracted draft is represented as

$$d_{ijk} = \langle \tilde{s}_{ijk}, e_{ijk}, \text{meta}(w_{ij}, D_i, \mathcal{T}) \rangle. \quad (13)$$

The extraction output for document D_i is

$$\mathcal{D}_i = \{d_{ijk} \mid I_{\text{proc}}(\tilde{s}_{ijk}) = 1\}. \quad (14)$$

3.3 Skill Management in the SkillBank

Skill extraction produces local drafts, while a deployable agent requires a coherent, non-redundant, and versioned SkillBank. ANYTHING2SKILL therefore performs skill management in three steps: document-level skill compilation, registry-level reconciliation, and visible skill-tree projection.

Document-level skill compilation. Window-level drafts may describe overlapping parts of the same capability. To produce canonical document-level skills, ANYTHING2SKILL groups drafts using a taxonomy-aware identity key. For a draft record d , let $s_d = \text{skill}(d)$ denote its skill payload and $D_d = \text{src}(d)$ denote its source record. Field accessors such as $a(\cdot)$, $v(\cdot)$, and $\tau_{\text{domain}}(\cdot)$ refer to fields of a skill contract.

$$\begin{aligned} \gamma(d) = (& \text{id}(D_d), a(s_d), g(s_d), v(s_d), \\ & \text{norm}(n(s_d)), \text{norm}(\mathbf{o}(s_d)), \tau_{\text{domain}}(s_d), \tau_{\text{family}}(s_d), \\ & \tau_{\text{task}}(s_d), \tau_{\text{method}}(s_d), \tau_{\text{stage}}(s_d)). \end{aligned} \quad (15)$$

Drafts sharing the same key are treated as document-local descriptions of the same capability. For each group

$$G_c = \{d \mid \gamma(d) = c\}, \quad (16)$$

a compiler produces a canonical skill specification:

$$s_c = C_\theta(G_c, \mathcal{E}_{G_c}), \quad (17)$$

where

$$\mathcal{E}_{G_c} = \bigcup_{d \in G_c} \mathcal{E}(d). \quad (18)$$

The compiler is constrained to preserve the group-level asset type, granularity, and taxonomy node:

$$\forall d \in G_c, \quad a(s_c) = a(s_d), \quad g(s_c) = g(s_d), \quad v(s_c) = v(s_d). \quad (19)$$

It is also required to cite only evidence contained in $\mathcal{E}_{G_c}$:

$$\mathcal{E}(s_c) \subseteq \mathcal{E}_{G_c}. \quad (20)$$

Conceptually, compilation solves a constrained canonicalization objective:

$$s_c^* = \arg \max_s \left[\sum_{d \in G_c} \text{cover}(s, d) - \lambda \text{unsupported}(s, \mathcal{E}_{G_c}) - \mu \text{redundancy}(s) \right], \quad (21)$$

subject to Eq. 19. This objective encourages the canonical skill to preserve supported procedural content while avoiding unsupported or redundant claims.

Registry-level reconciliation. After compilation, each incoming skill is reconciled with the persistent SkillBank. For an incoming skill s , let $\mathcal{S}_{\text{reg}}$ denote the canonical skills currently stored in the registry. ANYTHING2SKILL retrieves related existing skills:

$$H_k(s) = \text{TopK}_{t \in \mathcal{S}_{\text{reg}}} R(s, t), \quad (22)$$

where $R(s, t)$ is a hybrid retrieval score combining dense similarity, sparse lexical matching, and taxonomy-aware compatibility:

$$\begin{aligned} R(s, t) = & (1 - \alpha) \cos(\phi(s), \phi(t)) + \alpha \text{BM25}(x_s, x_t) \\ & + \beta_1 \mathbb{I}[a(s) = a(t) \wedge g(s) = g(t)] + \beta_2 \mathbb{I}[v(s) = v(t)] \\ & + \beta_3 \mathbb{I}[\ell(v(t)) < \ell(v(s))]. \end{aligned} \quad (23)$$

Here $\phi(\cdot)$ is the dense embedding of a metadata-rich skill text, and x_s, x_t are sparse retrieval texts built from skill names, descriptions, objectives, triggers, workflows, constraints, outputs, and taxonomy metadata.

The BM25 component is defined as

$$\text{BM25}(q, d) = \frac{1}{Z} \sum_{u \in q} \text{idf}(u) \cdot \frac{f(u, d)(k_1 + 1)}{f(u, d) + k_1(1 - b + b|d|/|d|)}, \quad (24)$$

where

$$\text{idf}(u) = \log \left(1 + \frac{N - n_u + 0.5}{n_u + 0.5} \right). \quad (25)$$

Here, the second BM25 argument d denotes a sparse retrieval text; the draft record variable in document-level compilation has local scope.

Given the retrieved neighbors $H_k(s)$, peer incoming skills $\mathcal{S}_{\text{peer}}(s)$, and support evidence $\mathcal{E}(s)$, a management model predicts a lifecycle action:

$$\eta = M_\theta(s, H_k(s), \mathcal{S}_{\text{peer}}(s), \mathcal{E}(s)), \quad (26)$$

where

$$\eta \in \{\text{CREATE, STRENGTHEN, REVISE, MERGE, SPLIT, UNCHANGED, DISCARD}\}. \quad (27)$$

The action space is taxonomy-constrained. For a selected existing skill $t \in H_k(s)$, update-like actions require taxonomy compatibility:

$$\begin{aligned} \eta \in & \{\text{STRENGTHEN, REVISE, MERGE, UNCHANGED}\} \\ \Rightarrow & a(s) = a(t) \wedge g(s) = g(t) \\ & \wedge (v(s) = v(t) \vee v(s) = \emptyset \vee v(t) = \emptyset). \end{aligned} \quad (28)$$

This prevents skills from being merged across incompatible asset layers, such as merging a safety rule into an executable micro-skill.

The registry applies the predicted action as follows. CREATE inserts a new skill. STRENGTHEN preserves the existing skill identity while adding new evidence and supported fields. REVISE updates the skill body when new evidence changes the procedure. MERGE combines compatible duplicate skills and deprecates secondary records. SPLIT creates or links a narrower child skill. UNCHANGED records provenance without changing the skill body. DISCARD removes unsupported or non-procedural candidates.

Each skill maintains a lifecycle state:

$$z(s) \in \{\text{candidate}, \text{draft}, \text{evaluating}, \text{active}, \text{watchlist}, \text{deprecated}, \text{retired}\}. \quad (29)$$

Versioned updates are recorded as

$$\text{ver}'(s) = U(\text{ver}(s), \eta, \Delta\mathcal{E}, \Delta s), \quad (30)$$

where $\Delta\mathcal{E}$ denotes newly attached evidence and Δs denotes the content change induced by the lifecycle action.

Visible skill-tree projection. The registry is the source of truth, while the visible skill tree is a rebuildable projection for navigation, inspection, and retrieval. After registry reconciliation, active skills are organized into a domain/family/level hierarchy according to $\mathcal{T}$. Let $\mathcal{S}_{\text{act}} \subseteq \mathcal{S}_{\text{reg}}$ denote the active canonical skills in the registry. For a skill s , we define its taxonomy level as

$$\text{level}(s) = \ell(v(s)). \quad (31)$$

For any skill s with $\text{level}(s) > 1$, candidate parents are selected from active canonical skills whose taxonomy nodes are valid parents of $v(s)$:

$$\mathcal{P}_{\text{parent}}(s) = \{t \in \mathcal{S}_{\text{act}} : \text{level}(t) = \text{level}(s) - 1 \wedge (v(t), v(s)) \in \mathcal{R}\}. \quad (32)$$

The parent is selected as

$$\text{parent}(s) = \arg \max_{t \in \mathcal{P}_{\text{parent}}(s)} R(s, t), \quad (33)$$

subject to a confidence threshold. If no compatible parent is found, the skill remains available in the registry but is left unlinked in the visible tree.

To reduce navigational redundancy, the system may further perform same-level consolidation over projected sibling skills. Two skills are considered consolidation candidates when they occupy the same metadata and taxonomy slot:

$$\kappa(s) = (a(s), g(s), v(s), \text{level}(s), \tau_{\text{domain}}(s), \tau_{\text{family}}(s), \tau_{\text{task}}(s), \tau_{\text{method}}(s), \tau_{\text{stage}}(s), \text{norm}(n(s))). \quad (34)$$

For each duplicate group

$$G_{\kappa} = \{s \in \mathcal{S}_{\text{act}} : \kappa(s) = \kappa\}, \quad (35)$$

the primary skill is selected as

$$s^* = \arg \max_{s \in G_{\kappa}} Q(s), \quad (36)$$

where $Q(s)$ ranks skills by lifecycle status, provenance coverage, content completeness, version maturity, and persistence in the registry. Non-conflicting fields are merged into the primary skill:

$$F(s^*) = \text{DedupUnion}_{s \in G_{\kappa}} F(s), \quad (37)$$

where F ranges over evidence records, invocation signals, contraindications, workflow steps, constraints, output contracts, triggers, tags, examples, and child links. Secondary records are deprecated, and visible-tree links are rewritten through

$$\psi(s) = \begin{cases} s^*, & \text{if } s \in G_{\kappa} \setminus \{s^*\}, \\ s, & \text{otherwise.} \end{cases} \quad (38)$$

For any skill-level hierarchy edge (s_p, s_c) , the updated edge becomes

$$(s_p, s_c) \mapsto (\psi(s_p), \psi(s_c)). \quad (39)$$

The final SkillBank is represented as

$$\mathcal{B} = \langle \mathcal{S}, \mathcal{E}_{\text{store}}, \mathcal{H}, \mathcal{V}_{\text{hist}}, \mathcal{L} \rangle, \quad (40)$$

where $\mathcal{S}$ is the set of canonical skills, $\mathcal{E}_{\text{store}}$ is the evidence store, $\mathcal{H}$ is the visible hierarchy projection, $\mathcal{V}_{\text{hist}}$ stores version history, and $\mathcal{L}$ stores lifecycle and provenance logs.

Table 1: Success rates on two command-line agent benchmarks. ANYTHING2SKILL improves agent performance by providing reusable procedural skills, and achieves the best results when combined with RAG.

Method	qsv	GitHub-CLI
Base Agent	81.60%	64.70%
RAG	95.41%	76.50%
ANYTHING2SKILL	91.95%	82.30%
ANYTHING2SKILL + RAG	98.85%	94.10%

3.4 Skill-Augmented Retrieval

At inference time, the constructed SkillBank can be combined with a standard RAG pipeline. Given a task query q , the agent retrieves both task-time evidence passages and relevant procedural skills:

$$\mathcal{C}_q = \text{Retrieve}_{\text{RAG}}(q, \mathcal{K}), \quad \mathcal{S}_q = \text{Retrieve}_{\text{Skill}}(q, \mathcal{B}). \quad (41)$$

The downstream agent then solves the task conditioned on both sources:

$$y = A_\theta(q, \mathcal{C}_q, \mathcal{S}_q). \quad (42)$$

Here, RAG provides task-specific declarative evidence, while ANYTHING2SKILL provides reusable procedural memory distilled from previous knowledge records.

4 Experiments

4.1 Experimental Setup

We evaluate ANYTHING2SKILL by asking two research questions. First, can compiled skill contracts improve agent performance even without task-time document retrieval? Second, can compiled skills complement retrieval-augmented generation (RAG) when both procedural skills and source passages are available [Lewis et al., 2020]? To answer these questions, we conduct experiments on two command-line agent benchmarks and report task success rate as the main evaluation metric.

Benchmarks. We use two command-line benchmarks that require agents to understand tool documentation, compose commands, and execute multi-step operations. The qsv benchmark focuses on CSV command-line operations grounded in the official qsv documentation [DatHere, 2026]. Tasks in this benchmark require the agent to select appropriate commands, compose options, and perform tabular-data transformations. The GitHub-CLI benchmark is constructed from GitHub CLI repository operation guides and gh command documentation [GitHub, 2026]. It covers 110 source documents and evaluates repository-management operations such as issue handling, pull-request workflows, release management, and repository inspection. From this corpus, ANYTHING2SKILL compiles 179 GitHub-CLI skills, including 1 global skill, 6 top-level skills, 62 second-level skills, and 110 micro-skills.

Compared configurations. We compare four context configurations. **Base Agent** denotes the setting where the agent receives no additional external context. **ANYTHING2SKILL** provides the agent with compiled skills retrieved from the SkillBank, but does not retrieve task-time source passages. **RAG** retrieves the top-3 source entries for each task from the original knowledge base. **ANYTHING2SKILL + RAG** combines both sources, where RAG provides task-specific declarative evidence and the SkillBank provides reusable procedural guidance. All configurations use the same base model, gpt5.4-mini, and are evaluated under the same task protocol.

Metric. We report benchmark success rate, defined as the percentage of tasks for which the agent successfully completes the requested command-line operation. This metric directly reflects whether the agent can translate a natural-language task into a valid and executable command-line solution.

4.2 Experimental Analysis

Compiled skills improve agents without task-time retrieval. As shown in Table 1, ANYTHING2SKILL substantially improves over the Base Agent on both benchmarks. On qsv, the success rate increases from 81.60% to 91.95%, yielding an absolute gain of 10.35 percentage points. On GitHub-CLI, the improvement is larger, from 64.70% to 82.30%, corresponding to an absolute gain of 17.60 percentage points. These results show that the compiled SkillBank is not

merely an additional retrieval index, but provides reusable procedural knowledge that can guide the agent even when no task-time document passages are retrieved.

Skills and RAG provide complementary context. RAG alone achieves strong performance on qsv, improving the Base Agent from 81.60% to 95.41%. This suggests that qsv tasks often benefit from directly retrieving command documentation and option descriptions. However, adding ANYTHING2SKILL on top of RAG further improves the success rate to 98.85%, indicating that procedural skill contracts provide useful guidance beyond raw source passages. The same complementarity is even more pronounced on GitHub-CLI: RAG alone reaches 76.50%, while ANYTHING2SKILL + RAG achieves 94.10%. This demonstrates that declarative evidence and procedural skills address different aspects of agent execution. RAG helps the agent access task-specific facts, while ANYTHING2SKILL helps the agent reuse structured procedures such as invocation conditions, action sequences, constraints, and output expectations.

Procedural skills are especially useful for complex CLI workflows. The gain from ANYTHING2SKILL is larger on GitHub-CLI than on qsv. One possible reason is that GitHub-CLI tasks involve more diverse repository-management workflows, where the agent must identify the appropriate command family, satisfy command-specific constraints, and compose multiple options under different scenarios. In such cases, simply retrieving documentation may expose relevant fragments, but the agent still needs to infer the correct procedure from them. By contrast, ANYTHING2SKILL compiles these latent procedures into structured skill contracts before inference, reducing the burden of task-time procedural reconstruction. This explains why ANYTHING2SKILL alone outperforms RAG alone on GitHub-CLI, and why their combination achieves the strongest result.

Overall effectiveness. Across both benchmarks, ANYTHING2SKILL + RAG consistently achieves the best performance, with 98.85% success on qsv and 94.10% success on GitHub-CLI. Compared with the Base Agent, the combined setting improves success rate by 17.25 percentage points on qsv and 29.40 percentage points on GitHub-CLI. These results support the central hypothesis of this paper: retrieval-augmented agents should not only retrieve external documents as declarative evidence, but also compile the procedural knowledge latent in those documents into reusable skills. The resulting SkillBank complements RAG and enables agents to move from knowledge access toward capability reuse.

5 Conclusion

In this paper, we introduced ANYTHING2SKILL, a taxonomy-guided framework that extends retrieval-augmented agents from external knowledge access to reusable capability acquisition. Unlike conventional RAG systems that primarily retrieve fragmented declarative evidence at inference time, ANYTHING2SKILL compiles heterogeneous knowledge records, including documents, manuals, dialogues, logs, and trajectories, into structured, evidence-grounded skill contracts. These skills are further consolidated, reconciled, versioned, and organized in a persistent SkillBank, enabling agents to retrieve procedural guidance alongside task-specific source passages. Experiments on qsv and GitHub-CLI demonstrate that compiled skills substantially improve agent success, and that combining ANYTHING2SKILL with RAG achieves the best performance across both benchmarks. These results suggest that the procedural knowledge latent in existing knowledge bases can be transformed into reusable agent capabilities, offering a promising direction for building retrieval-augmented agents that not only access knowledge, but also accumulate, manage, and reuse skills over time.

References

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020.
- Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021.

- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.
- Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 2023.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the 12th International Conference on Learning Representations*, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP. *arXiv preprint arXiv:2212.14024*, 2022.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. Autoskill: Experience-driven lifelong learning via skill self-evolution, 2026. URL <https://arxiv.org/abs/2603.01145>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations*, 2023a.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, 2024.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations*, 2024.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *arXiv preprint arXiv:2209.07753*, 2022.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *arXiv preprint arXiv:2305.16291*, 2023.
- Danqi Chen, Adam Fisch, Jason Weston, and Antoine Bordes. Reading wikipedia to answer open-domain questions. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017.
- Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. Latent retrieval for weakly supervised open domain question answering. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

- Minjoon Seo, Jinhyuk Lee, Tom Kwiatkowski, Ankur Parikh, Ali Farhadi, and Hannaneh Hajishirzi. Real-time open-domain question answering with dense-sparse phrase index. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4430–4441, 2019.
- Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. REPLUG: Retrieval-augmented black-box language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, et al. Kilt: a benchmark for knowledge intensive language tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2523–2544, 2021.
- Stephen Robertson and Hugo Zaragoza. *The probabilistic relevance framework: BM25 and beyond*, volume 4. Now Publishers Inc, 2009.
- Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. In *Transactions on Machine Learning Research*, 2022.
- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023a.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Ben Chess, and John Schulman. Webgpt: Browser-assisted question-answering with human feedback. In *arXiv preprint arXiv:2112.09332*, 2021.
- Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, Dor Muhlga, Noam Rozen, Erez Schwartz, Gal Shachaf, Shai Shalev-Shwartz, Amnon Shashua, and Moshe Tenenholz. MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. In *arXiv preprint arXiv:2205.00445*, 2022.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems*, 2023.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 3102–3116, 2023a.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023b.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. In *Transactions on Machine Learning Research*, 2023.
- Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in neural information processing systems*, 36: 51991–52008, 2023b.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023b.
- Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models. In *arXiv preprint arXiv:2207.05608*, 2022a.

- Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *Proceedings of the IEEE International Conference on Robotics and Automation*, 2023.
- Dhruv Shah, Błażej Osiniński, Sergey Levine, et al. Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action. In *Conference on robot learning*, pages 492–504. pmlr, 2023.
- Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. PaLM-E: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alex Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael S. Ryoo, Gabriela Salazar, Pannag Sanketi, Pierre Sermanet, Jaspier Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *arXiv preprint arXiv:2307.15818*, 2023.
- Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the 39th International Conference on Machine Learning*, 2022b.
- Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. In *Proceedings of the 7th International Conference on Learning Representations*, 2019.
- Karl Pertsch, Youngwoon Lee, and Joseph J. Lim. Accelerating reinforcement learning with learned skill priors. In *Proceedings of the 5th Conference on Robot Learning*, 2021.
- Jacob Andreas, Dan Klein, and Sergey Levine. Modular multitask reinforcement learning with policy sketches. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pages 835–850, 2021.
- Pratyusha Sharma, Antonio Torralba, and Jacob Andreas. Skill induction and planning with latent language. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- DatHere. qsv: Csv data wrangling toolkit. <https://qsv.dathere.com/>, 2026. Accessed: 2026-06-08.
- GitHub. Github cli manual. <https://cli.github.com/manual/>, 2026. Accessed: 2026-06-08.